Repair ledger — Histor-lite

Issues found and fixed during the build of Histor-lite, in Project Witness's
"make claims, attack claims, repair claims, narrow claims" spirit. Newest first.

R3 — HARDENING + DOCS (v0.1.1 → v0.1.2): duplicate-key ambiguity + stale wording

Same family as R2's findings — a parser that silently let a later value win.

Issue 1 (integrity ambiguity): duplicate checksum keys. `checksums.txt` was
parsed with a "last line wins" rule, so a bogus `source_run_sha256` line followed
by the correct one still verified OK (the tampered line hid behind the correct
duplicate). Fix: `parse_checksums` now rejects ANY duplicate key — each
recognised key must appear exactly once — raising a `CaseError`. This covers
`source_run_sha256`, `case_summary_sha256`, and

`sentinel_replay_report_sha256`.
All existing checks (source-run consistency, case-summary hash, replay-report
hash, canonical form) are unchanged.
(`histor_lite/case_summary.py`.)

Issue 2 (stale wording): release_check identity strings. `release_check.py`
still said "Histor-lite v0.1" in its header and banner and hinted at
`histor_lite_v01.sha256`. Updated current-identity strings to v0.1.2 and the
checksum hint to `histor_lite_v012.sha256`. A tree-wide sweep also bumped the
remaining current-identity strings (selfcheck/CLI banners, README,
KNOWN_LIMITATIONS, RUN_FORMAT, package headers) to v0.1.2. Historical references
to v0.1.0 and v0.1.1 in this ledger are left intact as history.

Tests added (37 total, was 33): duplicate `source_run_sha256`,
`case_summary_sha256`, and `sentinel_replay_report_sha256` each fail; the exact
bogus-then-correct duplicate attack fails; valid case still verifies; the
source-run, case-summary-hash, replay-report-hash,

and non-canonical checks all
still fail on tamper.

Scope: parser hardening + version-string cleanup only. No new features, no
signing, no anchoring, no new component imports, no change to Sentinel.
`HISTOR_LITE_VERSION` -> 0.1.2.
Verified: all release gates pass from a clean extraction; 37 tests under
normal and `-O`; selfcheck 8/8 both modes; the optional real-Sentinel
integration test still passes against the frozen v0.2.2 build.

R2 — CORRECTNESS REPAIR (v0.1.0 → v0.1.1): two review findings

Two issues found in review, both the same root mistake — trusting one channel of
a result while ignoring a corroborating one.

Issue 1 (release blocker): inconsistent Sentinel verifier result imported.
A verifier returning `verdict="PASS_STRICT"` with `ok=False` was imported as
`IMPORTED` (and `sentinel_replay_ok: false` was written into the summary) — a
self-contradictory case. Fix: an accepted verdict

(`PASS_STRICT` / `PASS_WITH_ENVIRONMENTAL_EXCEPTIONS`) is honoured only when `ok is True`. An accepted-looking verdict with `ok=False` is refused with the new machine-stable reason `SENTINEL_VERDICT_INCONSISTENT`. `FAIL` and `REPLAY_INCOMPATIBLE` remain refused regardless of `ok`. (`histor_lite/sentinel_import.py`.)

Issue 2 (integrity gap): verify-case ignored the source_run_sha256 line. Modifying only the `source_run_sha256` line in `checksums.txt` still reported `CASE INTEGRITY OK`. The source run file is not copied into the case folder, so it cannot be rehashed — but its recorded checksum metadata must be internally consistent. Fix: `verify-case` now requires `source_run_sha256` in `checksums.txt`, requires it to be a well-formed 64-hex digest, and requires it to equal the `source_run_sha256` embedded in `case_summary.json`. Missing, malformed, or mismatched → fail. The existing case-summary-hash, replay-report-hash, and canonical-form checks are unchanged. (`histor_lite/case_summary.py`.)

Tests added (33 total, was 23):
PASS_STRICT/PASS_WITH_ENVIRONMENTAL_EXCEPTIONS with `ok=False` refused;
FAIL/REPLAY_INCOMPATIBLE with `ok=True` still refused;
both PASS verdicts with `ok=True` still import;
source_run_sha256 value-tamper,
missing-line, and malformed-digest all detected;
summary-hash and report-hash
checks still effective. A new selfcheck gate `refuse_inconsistent_verdict`
guards Issue 1 in-process (now 8 gates).

Scope: correctness repair only. No new features, no signing, no anchoring, no
new component imports, no change to Sentinel. `HISTOR_LITE_VERSION` -> 0.1.1.
Verified: all release gates pass from a clean extraction; 33 tests under
normal and `-O`; selfcheck 8/8 both modes; the optional real-Sentinel
integration test still passes against the frozen v0.2.2 build.

R1 — FIRST BUILD (v0.1.0): Histor-lite importer/verifier

First Histor-lite build — Move 4 after freezing Sentinel Core v0.2.2. A narrow

importer/verifier that takes a Sentinel v0.2.2 run file, verifies it with the
frozen Sentinel verifier, and writes a stack-level case summary. This is the
first connection between Sentinel and the wider Project Witness stack.

Scope held deliberately narrow: Sentinel run -> Histor-lite import ->
verified case summary. Nothing else. No signing, no external anchoring, no
Aletheia/ODIN/Atlas/Companion import, no GUI, no LLM integration, no full Histor.
Sentinel Core is not modified, re-versioned, or extended.

Design decisions worth recording:

- **Verifier invoked in-process, not via subprocess.** Histor-lite inserts the
  Sentinel root on `sys.path` and calls `sentinel_core.replay.replay_from_path`
  directly, reading the exact `.verdict` / `.report`. This avoids stdout parsing
  and the subprocess-pipe hang class that Sentinel itself hit (Sentinel
  REPAIR_LEDGER R12/R14). A verifier exception is caught and converted to
  `VerifierUnavailable`, satisfying "if Sentinel replay cannot be run: refuse".

- **Module-cache eviction at the verifier boundary.** Python caches imported modules, so a second Sentinel root would silently reuse the first. Before importing, the verifier evicts any cached `sentinel_core*` modules so it loads the verifier at the intended root. This keeps stub-based tests and the optional real-Sentinel test from contaminating each other.
- **Verdict preserved exactly.** A `PASS_WITH_ENVIRONMENTAL_EXCEPTIONS` run is recorded as exactly that and mapped to `IMPORTED_WITH_ENVIRONMENTAL_EXCEPTIONS`; it is never relabelled as strict. Sentinel's disclosed limits (unkeyed hashes; environmental ERROR structurally verified) are carried verbatim into every case summary so a downstream reader is not misled.
- **Honest, unkeyed integrity only.** `verify-case` proves internal consistency of the case folder (recomputed SHA-256 vs `checksums.txt`, plus a canonical-form re-check of the summary). It does not pretend to be tamper-proof; whole-file reconstruction is out of scope (KNOWN_LIMITATIONS H1/H2), exactly as for Sentinel.

- **Tests use a stub Sentinel root** (a minimal `sentinel_core/replay.py` whose verdict is driven by the run file) so Histor-lite's import/refuse/case logic is tested deterministically for every verdict without an external Sentinel checkout. An optional integration test drives the **real** frozen Sentinel when `SENTINEL_V022_ROOT` is set, and skips otherwise.

Self-fix during build: the first `deterministic_canonical_json` test wrongly asserted that two case summaries built from *different* run files would be byte-identical; they correctly differ in `source_run_path`. The test was corrected to import the *same* run file twice (same inputs -> identical bytes). The determinism guarantee (same inputs -> same canonical bytes) was never in question; the test premise was wrong.

Verified: all release gates pass from a clean extraction (see TEST_RESULTS.md); selfcheck 7/7 in-process gates; 23 unit tests pass under normal and `-O`; the

optional real-Sentinel integration test passes against the frozen v0.2.2 build.